

CIS 121 Assignment – Reading Data from a File. Practice Pass by Value Functions. For each problem develop an IPO within this document. Save the document and upload to Blackboard for grading. Then develop a C++ program for each problem and upload the .cpp file to Blackboard.

1. Read in auto make, model, MSRP (manufacturer suggested resale price) and sales price from a file. See auto.txt below. For each auto (line of data) compute the savings to be MSRP – sales price and sales tax in a single function.

Pass to the function MSRP, sales price by value, sales price and sales tax by reference. The function will:

1. Compute savings = MSRP – Sales Price
2. Compute sales tax = Sales Price * 0.07

For each auto (line of data) display make, model, MSRP, sales price, savings and sales tax. Sum the savings.

After all autos (lines of data processed) display sum of savings.

Auto.txt

Honda Accord 25000.00 22000.00

Honda CRV 30000.00 28000.00

Toyota Corrola 28000.00 25000.00

Ford Fusion 23000.00 20000.00

Input	Process	Output
• Make • Model • MSRP • Sales Price	<i>computeAutoSale(MSRP & Sales Price [Ref: Savings & Sales Tax])</i> • Calculate Savings by finding the difference between MSRP and Sales Price • Calculate Sales Tax by finding the product of Sales Price * 7% (0.07)	<i>Each Entry</i> • Make • Model • MSRP • Sales Price • Savings • Sales Tax
	<i>Main()</i> • Declare Make, Model, MSRP, Sales Price, Savings, & Sales Tax • Declare Sum of Savings, set at 0 • Open Auto.txt • While not EOF • Get Make, Model, MSRP, & Sales Price • <u>Call <i>computeAutoSale</i></u> • Add Savings to Sum of Savings • Display Make, Model, MSRP, Sales Price, Savings, & Sales Tax • End Loop • Close Auto.txt	<i>After All Entries</i> • Sum of Savings

	• Display Sum of Savings	
--	--------------------------	--

2. Read in the grocery item, quantity and cost per item from a file. See grocery.txt below. For each item determine the extended price (quantity * cost per item) within a function.

Pass to the function quantity and price. The function will:

3. Compute extended price = quantity * price
4. Return extended price

For each item (line of data) display item, quantity, cost per item and extended price.

Also, sum the extended price.

After all lines of data are processed, compute tax of 7% (sum of extended price * 0.07).

Compute total receipt to be sum of extended price + tax. Display sum of extended price, tax and total receipt.

Grocery.txt

Banana	10	0.50
Oranges	5	0.10
Cookies	15	0.30
Hamburgers	4	1.00
Buns	4	0.50
Soda	10	0.99

Input	Process	Output
• Grocery Item • Quantity • Item Cost	<i>computeOrderCost(Quantity & Item Cost [Ref: Extended Price])</i> • Calculate Extended Price by finding the product of Quantity and Item Cost	<i>Each Entry</i> • Grocery Item • Quantity • Item Cost • Extended Price
	<i>Main()</i> • Declare Grocery Item, Quantity, Item Cost, Extended Price, Tax, & Total Receipt • Declare Sum of Extended Price, set at 0 • Open Grocery.txt • While not EOF • Get Grocery Item, Quantity, & Item Cost • <u>Call <i>computeOrderCost</i></u> • Add Extended Price to Sum of Extended Price • Display Grocery Item, Quantity, Item Cost, & Extended Price	<i>After All Entry</i> • Sum of Extended Price • Tax • Total Receipt

	• End Loop • Close Grocery.txt • Calculate Tax by finding the product of Sum of Extended Price and 7% (0.07) • Calculate Total Receipt by finding the sum of Sum of Extended Price and Tax • Display Sum of Extended Price, Tax, & Total Receipt	
--	---	--

3. Read gallons of gas used and miles travelled from a file. See trips.txt below. For each trip compute miles per gallon within a function.

Pass to the function gallons and miles. Within the function:

1. Compute mpg = miles / gallons.
2. Return mpg.

For each trip (line of data) display gallons, miles and mpg.

Sum miles travelled and gallons of gas used.

After all data is processed (end of file) display sum of gallons and sum of miles travelled.

Trips.txt

```
20    500
10    600
5     50
10    350
```

Input	Process	Output
• Gallons Used • Miles Travelled	<i>computeMpg(Gallons Used & Miles Travelled [Ref: Mpg])</i> • Calculate Mpg by finding the quotient of Miles Travelled and Gallons Used	<i>Each Entry</i> • Gallons Used • Miles Travelled • Mpg
	<i>Main()</i> • Declare Gallons Used, Miles Travelled, & Mpg • Declare Total Gallons Used & Total Miles Travelled, set at 0 • Open Trips.txt • While not EOF • Get Gallons Used & Miles Travelled • <u>Call computeMpg</u> • Add Gallons Used to Total Gallons Used • Add Miles Travelled to Total Miles Travelled	<i>After All Entries</i> • Total Gallons Used • Total Miles Travelled

	• Display Gallons Used, Miles Travelled, & Mpg • End Loop • Display Total Gallons Used and Total Miles Travelled	
--	---	--

4. Read in last name and annual salary from a file. See empl.txt below. For each employee (line of data) compute their bi-weekly pay within a function.

Pass to the function the annual salary. The function will:

1. Compute bi-weekly salary = annual salary / 26
2. Return bi-weekly salary

For each line display last name, annual salary and bi-weekly salary. Sum the annual salaries. Count the number of employees (each line of data).

After all employees processed compute average annual salary (total annual salary / count of employees). Display sum of annual salary, count of employees and average annual salary.

Empl.txt

Jones 50000.00

Adams 65000.00

Baker 45000.00

Smith 75000.00

Input	Process	Output
• Employee Last Name • Annual Salary	- computeBiWeeklyPay(Annual Salary [Ref: Bi-Weekly Salary]) - Calculate Bi-Weekly Salary by finding the quotient of Annual Salary and 26	<i>Each Entry</i> • Employee Last Name • Annual Salary • Bi-Weekly Salary
	Main() • Declare Employee Last Name, Annual Salary, & Bi-Weekly Salary • Declare Sum of Annual Salaries, Number of Employees, & Average Annual Salary, set at 0 • Open Empl.txt • While not EOF • Get Employee Last Name & Annual Salary • <u>Call computeBiWeeklyPay</u> • Add Annual Salary to Sum of Annual Salary • Add 1 to Number of Employees	<i>After All Entries</i> • Sum of Annual Salaries • Number of Employees • Average Annual Salary

	• Display Employee Last name, Annual Salary, & Bi-Weekly Salary • End Loop • Calculate Average Annual Salary by finding the quotient Sum of Annual Salaries & Number of Employees • Display Sum of Annual Salaries, Number of Employees, & Average Annual Salary	
--	--	--

5. The input to this program is last name, student code and credits taken. See file student.txt below.

You are to read each line of data into your program (one line at a time of course). Write a function to calculate tuition owed and course fees in a single function. You must pass to the function student code and credits taken by value and tuition owed and course fees by reference.

Within the function compute tuition owed and course fees:

1. Determine cost per credit hour. In district students (student code == 'I') pay \$250.00 per credit hour. Out of district students (student code == 'O') pay \$500.00 per credit hour.
2. Calculate tuition owed to be credit hours * cost per credit hour
3. Compute course fees as 10% of the total tuition.

Also, within the program sum the tuition owed for all students. Count the number of students. For each student (line of data), display last name, student code, cost per credit hour, credits taken, tuition owed and course fees. Additionally, sum the tuition owed.

After all students are processed (end of loop), calculate the average tuition cost per student (sum of tuition owed / number of students). Display sum of tuition owed, number of students and average tuition cost per student.

Student.txt

Jones	I	15.00
Baker	O	20.00
Davis	I	10.00
Michaels	O	12.00
Baez	I	12.00

Input	Process	Output
• Student's Last Name • District code • Credit Hours	<i>computeClassCost(District Code & Credit Hours [Ref: Tuition owed & Course Fees])</i> • If District Code equals I, set Cost/Credit Hour at 250.00	<i>Each Entry</i> • Student Last Name • District Code • Credit Hours

	○ If District Code equals O, set Cost/Credit Hour at 500.00 ○ Otherwise, End Program • Calculate Tuition Owed by finding the product of Credit Hours and Cost/Credit Hour • Calculate Course Fees by finding the product of Tuition Owed and 10% (0.1)	• Cost/Credit Hour • Tuition Owed • Course Fees
	<i>Main()</i> • Declare Student's Last Name, District code, Credit Hours, Cost/Credit Hour, Tuition owed, Course fees, & Average Tuition • Declare Sum of Tuition Owed & Number of Students, set at 0 • Open Student.txt • While not EOF • Get Student's Last Name, District Code, & Credit Hours • <u>Call <i>computeClassCost</i></u> • Add Tuition Owed to Sum of Tuition Owed • Add 1 to Number of Students • Display Student Last Name, District Code, Credit Hours, Cost/Credit Hour, Tuition Owed, Course Fees • End Loop • Calculate Average Tuition by finding the quotient of Sum of Tuition Owed and Number of Students • Display Sum of Tuition Owed, Number of Students, & Average Tuition	<i>After All Entries</i> • Sum of Tuition Owed • Number of Students • Average Tuition